Session 05 - Accounts, Login and Document Search - Tests

Python Contoso Vault - Session 05 of 07

Part 1 - Core part A: accounts.py

Goal

Keep the user accounts of Contoso's document vault. Store a fingerprint of each password, called a hash, and never the password itself. To check a login, hash the password the user types and compare the result with the stored hash.

Open the Python shell in the `05-accounts-login-and-document-search` folder and run `from accounts import hash_password, create_account, check_login`, then make each call in the test plan.

Instructions

1. Import the `hashlib` module.
2. Define a function `hash_password` that takes a password as a string.
   2.1. Hash the password with SHA-256.
   2.2. Return the hash as a hex string.
3. Define a function `create_account` that takes the accounts as a dictionary, a username as a string and a password as a string.
   3.1. Return `False` if the accounts already hold the username.
   3.2. Otherwise, add the username to the accounts with the hash of the password.
   3.3. Return `True`.
4. Define a function `check_login` that takes the accounts as a dictionary, a username as a string and a password as a string.
   4.1. Return `True` if the accounts hold the username and its stored hash equals the hash of the password.
   4.2. Return `False` otherwise.

Constraints

- The hash is SHA-256 over the password's UTF-8 bytes, written as 64 lowercase hex characters.
- The accounts dictionary maps each username to the hash of its password. The password itself is never stored. Accounts for `"nimali"` and `"kasun"` look like this:

```
{
    "nimali": "31ba6993336b8fbd391af9cac71bb3928077583c9c13c595ea61dfc3b7a576fa",
    "kasun": "82d5fb592cbf8dd48bfbbb9fa2c4075396353ef09f6ea53fae8bfefbba87a465",
}
```

- Usernames and passwords match exactly. `"Nimali"` and `"nimali"` are different usernames.
- `create_account` leaves the accounts unchanged when the username is taken.
- The functions neither print, read input nor write a file.
- Importing the file reads no input and prints nothing.

Design

The following rules say how to write the code. Follow them.

- Give every variable and function a type hint.

Input validation

Not required. Any username and password are valid, including empty ones.

Test plan

`nimali_hash` is `31ba6993336b8fbd391af9cac71bb3928077583c9c13c595ea61dfc3b7a576fa`, the hash of `"lotus-2026"`.

Likewise, `upper_hash` is `9c1848f006ad7f8890ed6f3183e1604a306a0796235070776df77c151a1c698d`, the hash of `"Lotus-2026"`.

Likewise, `empty_hash` is `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`, the hash of `""`.

Likewise, `kasun_hash` is `82d5fb592cbf8dd48bfbbb9fa2c4075396353ef09f6ea53fae8bfefbba87a465`, the hash of `"kandy-rain-88"`.

In tests 5 to 9, `accounts` starts as `{"nimali": nimali_hash}`.

Test	Call	Expected output	Actual output	Pass
1	`hash_password("lotus-2026")`	`nimali_hash`		
2	`hash_password("Lotus-2026")`	`upper_hash`		
3	`hash_password("")`	`empty_hash`		
4	`create_account(accounts, "nimali", "lotus-2026")`, with `accounts` as `{}`	`True`. `accounts` is `{"nimali": nimali_hash}`		
5	`create_account(accounts, "nimali", "kandy-rain-88")`	`False`. `accounts` is unchanged		
6	`create_account(accounts, "kasun", "kandy-rain-88")`	`True`. `accounts["kasun"]` is `kasun_hash`		
7	`check_login(accounts, "nimali", "lotus-2026")`	`True`		
8	`check_login(accounts, "nimali", "Lotus-2026")`	`False`		
9	`check_login(accounts, "Nimali", "lotus-2026")`	`False`		
10	`check_login({}, "nimali", "lotus-2026")`	`False`		
11	`import accounts`	Reads no input and prints nothing		

Done criteria

- [ ] Every test plan row gives exactly the expected result.

Part 2 - Core part B: vault.py

Goal

Make `vault.py` ask who the user is before it opens. Let the user sign up once with a username and a password, then log in each time the program starts. Also let the user search documents by ID, or by a word in a document's title.

Run `python vault.py` in the terminal from the `05-accounts-login-and-document-search` folder. Type each input line and press Enter after it. Delete `accounts.json` and `documents.json` before each test that starts in an empty folder. To try one function, open the Python shell in the same folder and run `from vault import search_documents`.

Provided

Copy your own `caesar.py` and `vault.py` from your Session 04 folder into this folder, and build `accounts.py` beside them. `vault.py` already defines `load_documents`, `save_documents`, `create_document` and `check_key`.

The program now opens with an account menu of the following three lines:

```
1. Sign up
2. Log in
3. Quit
```

The document menu now shows the following five lines:

```
1. Create
2. List
3. Read
4. Search
5. Quit
```

Instructions

Amend the copied `vault.py`. Keep `load_documents`, `save_documents`, `create_document` and `check_key` as they are, and add these steps to the same file.

1. Import `create_account` and `check_login` from `accounts.py`.
2. Define a function `search_documents` that takes the documents as a list and a search text as a string.
    2.1. For each document, in ID order:
        2.1.1. Keep the document if its ID, as a string, equals the search text, or its title contains the search text, ignoring case.
    2.2. Return the kept documents as a list of dictionaries.
3. When the file runs as a program:
    3.1. Load the accounts from `accounts.json` at the start, as a dictionary. Start the accounts as an empty dictionary if the file does not exist.
    3.2. Repeat until the user logs in or quits:
        3.2.1. Print the account menu.
        3.2.2. Read a choice as a string.
        3.2.3. On `1`:
            3.2.3.1. Read a username and a password.
            3.2.3.2. Create the account.
            3.2.3.3. If the account was created, save the accounts to `accounts.json` and print `Account created.`
            3.2.3.4. Otherwise, print `Username taken.`
        3.2.4. On `2`:
            3.2.4.1. Read a username and a password.
            3.2.4.2. If the login is right, print `Welcome, NAME.`, where `NAME` is the username, and go on to the document menu.

        3.2.4.3. Otherwise, print `Wrong username or password.`
    3.2.5. On `3`, print `Goodbye.` and end the program.
    3.2.6. On any other choice, print `Invalid choice.`
3.3. Print the five-line document menu in place of the Session 04 four-line document menu.
3.4. On `4`:
    3.4.1. Read a search text as a string.
    3.4.2. Search the documents for the search text.
    3.4.3. Print one line `ID. Title` per matching document, in ID order.
    3.4.4. Print `No matches.` if no document matches.
3.5. On `5`, print `Goodbye.` and end the program.
3.6. Keep every other step of the Session 04 program.

Constraints

- Every constraint in the Session 04 `vault.py` test plan still holds, with the five-line document menu in place of the Session 04 four-line document menu.
- The account menu prints before every account choice, as the exact three lines above.
- Read each value with a prompt on the same line, as in `input("Title: ")`. Each prompt is its label, a colon and one trailing space. The prompts are:
  - `"Choice: "` after the account menu lines and after the document menu lines.
  - `"Username: "` then `"Password: "` at Sign up and Log in.
  - `"Search text: "` at Search.
  - The Create and Read prompts, `"Title: "`, `"Body: "`, `"Key: "` and `"ID: "`, as in Session 03.
- After a retry message, the prompt that asked for the value prints again.
- Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- Sign up and log in each read the username, then the password.
- `accounts.json` sits in the folder the program runs from. It holds one JSON object of the accounts as `create_account` stores them. It never holds a password.
- The program writes `accounts.json` only when an account is created.
- A wrong username and a wrong password print the same message.
- The search text is used exactly as typed. An empty search text matches every document. A document that matches by both ID and title appears once.
- `search_documents` neither prints nor reads input.
- Importing the file reads no input, prints nothing and writes no file.

Design

The following rules say how to write the code. Follow them.

- Every Design point in the Session 04 `vault.py` test plan still applies.
- Give every variable and function a type hint.

Input validation

Required for every key and ID, as in Session 04. Not required for usernames, passwords and search texts. Any text is valid, including an empty line. Assume `accounts.json` is missing or was written by this program.

Test plan

Function tests

`document_1` and `document_2` are the Session 04 documents. `document_3` is `{"id": 3, "title": "Galle rent", "body": "Lfqqj xmtu wjsy nx Wx 90,000.", "check": "HTSYTXT-AFZQY"}`. Here, `documents` is `[document_1, document_2, document_3]`. `document_4` is `{"id": 4, "title": "Floor 4 lease", "body": "Msvvy 4 slhzl lukz pu Thyjo.", "check": "JVUAVZV-CHBSA"}`.

Test	Call	Expected output	Actual output	Pass
1	`search_documents(documents, "rent")`	`[document_2, document_3]`		
2	`search_documents(documents, "RENT")`	`[document_2, document_3]`		
3	`search_documents(documents, "pay")`	`[document_1]`		
4	`search_documents(documents, "kandy rent")`	`[document_2]`		
5	`search_documents(documents, "2")`	`[document_2]`		
6	`search_documents(documents, "Colombo")`	`[]`		
7	`search_documents(documents, "")`	`[document_1, document_2, document_3]`		
8	`search_documents([], "rent")`	`[]`		
9	`search_documents([document_4], "4")`	`[document_4]`		

Program tests

Each test runs `python vault.py` in an empty folder, once or twice in a row. "Sign up Nimali" stands for the three input lines `"1"`, `"nimali"`, `"lotus-2026"`, and "Log in Nimali" for `"2"`, `"nimali"`, `"lotus-2026"`. "Create Payroll" and "Create Kandy rent" are as in Session 04. "Create Galle rent" stands for `"1"`, `"Galle rent"`, `"Galle shop rent is Rs 90,000."`, `"5"`. `nimali_hash` is as in Core part A. Every run prints the account menu and the prompt `"Choice: "` before each account choice, and the document menu and the prompt `"Choice: "` before each document choice. Every run ends with the `Goodbye.` line. The Expected output cell lists only the lines of the last run that are not menu lines, prompts or `Goodbye.`. It then gives `accounts.json` after the last run.

Test	First run input lines	Second run input lines	Expected output	Actual output	Pass
10	`3`	None	No lines except the menus and `Goodbye.` File: none		
11	Sign up Nimali, `3`	None	`Account created.` File: `{"nimali": nimali_hash}`		
12	Sign up Nimali, `1`, `nimali`, `kandy-rain-88`, `3`	None	`Account created.`, `Username taken.` File: `{"nimali": nimali_hash}`		
13	Sign up Nimali, Log in Nimali, `5`	None	`Account created.`, `Welcome, nimali.` File: `{"nimali": nimali_hash}`		
14	`2`, `nimali`, `lotus-2026`, `3`	None	`Wrong username or password.` File: none		
15	Sign up Nimali, `2`, `nimali`, `Lotus-2026`, `2`, `Nimali`, `lotus-2026`, Log in Nimali, `5`	None	`Account created.`, `Wrong username or password.` twice, `Welcome, nimali.` File: `{"nimali": nimali_hash}`		
16	Sign up Nimali, `3`	Log in Nimali, `5`	`Welcome, nimali.` File: `{"nimali": nimali_hash}`		
17	`4`, `x`, empty line, `3`	None	`Invalid choice.` three times File: none		
18	Sign up Nimali, Log in Nimali, `6`, `5`	None	`Account created.`, `Welcome, nimali.`, `Invalid choice.` File: `{"nimali": nimali_hash}`		
19	Sign up Nimali, Log in Nimali, Create Payroll, Create Kandy rent, Create Galle rent, `4`, `rent`, `4`, `PAY`, `4`, `Colombo`, `5`	None	`Account created.`, `Welcome, nimali.`, `Saved document 1.`, `Saved document 2.`, `Saved document 3.`, `2. Kandy rent`, `3. Galle rent`, `1. Payroll`, `No matches.` File: `{"nimali": nimali_hash}`		
20	Sign up Nimali, Log in Nimali, Create Payroll, Create Kandy rent, `5`	Log in Nimali, `4`, `2`, `3`, `2`, `11`, `5`	`Welcome, nimali.`, `2. Kandy rent`, `Kandy office rent is Rs 150,000.` File: `{"nimali": nimali_hash}`		
21	Sign up Nimali, Log in Nimali, `4`, `rent`, `5`	None	`Account created.`, `Welcome, nimali.`, `No matches.` File: `{"nimali": nimali_hash}`		
22	`import vault`	None	Reads no input and prints nothing File: none		
23	Each Session 04 program test, 7 to 13 and 15, with Sign up Nimali and Log in Nimali before the first run's input lines, Log in Nimali before the second run's input lines, and `5` in place of each document menu choice `4`	As in the Session 04 test with the same number	`Welcome, nimali.`, then the Session 04 expected output lines. A one-run test starts with `Account created.` File: not checked		

In every program test, the program ends after `Goodbye.` and shows no Python error message.

Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python vault.py`.

Part 3 - Core part C: crack.py

Goal

An old Contoso system stored each password as a plain SHA-256 hash. Show why that is not safe. Hash every word in a list of common passwords, then look up each user's stored hash. Find every user who picked a common password.

Run `python crack.py` in the terminal from the `05-accounts-login-and-document-search` folder. To try one function, open the Python shell in the same folder and run `from crack import crack`.

Provided

`users_legacy.json` and `common_passwords.txt` arrive in this folder when you sync your fork.

- `users_legacy.json` holds 50 accounts in the same shape as `accounts.json`: one JSON object that maps each username to the SHA-256 hex hash of its password. A file with only its first two accounts would hold:

```
{
  "nimali": "48c05e3c16c9c2bca6f3557577ac56b73efbe335d9fad22c740ab4165eef03a8",
  "kasun": "8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92"
}
```

- `common_passwords.txt` holds 100 common passwords, one per line. It starts with `"123456"`, `"password"`, `"12345678"` and `"qwerty"`.

Instructions

1. Import the `json` module, and `hash_password` from `accounts.py`.
2. Define a function `crack` that takes the users as a dictionary and the path of the wordlist text file as a string.
   2.1. Build the table, a dictionary that maps the hash of each word in the wordlist to the word.
   2.2. For each user, in order:
      2.2.1. If the table holds the user's stored hash, add the username and the word to the cracked dictionary.
   2.3. Return the cracked dictionary.
3. When the file runs as a program:
   3.1. Load the users from `users_legacy.json`, as a dictionary.
   3.2. Crack the users with `common_passwords.txt`.
   3.3. Print one line `username: password` per cracked user, in file order.
   3.4. Print `N of M accounts cracked.`, where `N` is the number cracked and `M` the number of users.

Constraints

- Each word is the text of one line, without its line ending.
- The cracked dictionary maps each cracked username to its password. Its keys follow the order of the users. A user whose hash is not in the table is left out.
- Two users with the same password are both cracked.
- `crack` neither prints nor reads input.
- Importing the file reads no input and prints nothing.

Design

The following rules say how to write the code. Follow them.

- Hash each word in the wordlist once, whatever the number of users.
- Give every variable and function a type hint.

Input validation

Not required. Assume both files exist, `users_legacy.json` holds the shape above, and `common_passwords.txt` has no empty lines and no repeated words.

Test plan

Function tests

Each test runs in a folder that holds a copy of `common_passwords.txt`. `legacy_users` is the dictionary loaded from `users_legacy.json`. `nimali_hash` is as in Core part A.

Test	Call	Expected output	Actual output	Pass
1	`crack(legacy_users, "common_passwords.txt")`	`{"kasun": "123456", "dilani": "password", "chamara": "qwerty", "nuwan": "iloveyou", "roshan": "srilanka123", "chathura": "colombo1", "asela": "sunshine", "dasun": "password123", "lakmal": "password123", "supun": "dragon", "ravindu": "letmein", "pasindu": "cricket"}`		
2	`crack({"nimali": nimali_hash}, path)`, where `path` names a wordlist with the one line `lotus-2026`	`{"nimali": "lotus-2026"}`		
3	`crack({"nimali": nimali_hash}, path)`, where `path` names a wordlist with the one line `Lotus-2026`	`{}`		
4	`crack({}, "common_passwords.txt")`	`{}`		
5	`crack(legacy_users, path)`, where `path` names an empty wordlist	`{}`		

Program tests

Test	Command	Expected output	Actual output	Pass
6	`python crack.py`	`kasun: 123456`, `dilani: password`, `chamara: qwerty`, `nuwan: iloveyou`, `roshan: srilanka123`, `chathura: colombo1`, `asela: sunshine`, `dasun: password123`, `lakmal: password123`, `supun: dragon`, `ravindu: letmein`, `pasindu: cricket`, `12 of 50 accounts cracked.`		
7	`import crack`	Reads no input and prints nothing		

In every program test, the program shows no Python error message.

Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python crack.py`.

Part 4 - Challenge: passwords.py

Goal

Fix the weakness that `crack.py` showed. Add a random salt to each password before hashing, so two users with the same password get different hashes. Hash with scrypt, which is slow on purpose, so every guess costs an attacker time. Move an old account to the new form the next time its owner logs in.

Open the Python shell in the `05-accounts-login-and-document-search` folder and run `from passwords import hash_password, verify_password, login`, then make each call in the test plan.

Instructions

1. Import the `hashlib`, `hmac` and `os` modules.
2. Define a function `hash_password` that takes a password as a string.
   2.1. Make a fresh random salt of 16 bytes.
   2.2. Hash the password with scrypt, using the salt, `n` 16384, `r` 8, `p` 1 and a 64-byte hash.
   2.3. Return the record as a dictionary with the keys `algo`, `salt`, `n`, `r`, `p` and `hash`.
3. Define a function `verify_password` that takes a password as a string and a scrypt record as a dictionary.
   3.1. Hash the password with scrypt, using the salt, `n`, `r` and `p` stored in the record.
   3.2. Return `True` if the new hash equals the record's hash, and `False` otherwise.
4. Define a function `login` that takes a username as a string, a password as a string and the users as a dictionary.
   4.1. Return `False` if the users do not hold the username.
   4.2. If the user's record is a legacy record:
      4.2.1. Return `False` if the SHA-256 hex hash of the password does not equal the record's hash.
      4.2.2. Otherwise, replace the user's record with a new scrypt record of the password, and return `True`.
   4.3. Otherwise, return whether the password verifies against the user's scrypt record.

Constraints

- A scrypt record holds `"algo": "scrypt"`, the salt as 32 hex characters, `n`, `r` and `p` as integers, and the hash as 128 hex characters.
- A legacy record is `{"algo": "sha256", "hash": <hex>}`, where the hash is the SHA-256 hex hash of the password's UTF-8 bytes.
- The users dictionary maps each username to a record of either kind. Users with a scrypt record for `"nimali"` and a legacy record for `"kasun"` look like this:

```
{
    "nimali": {
        "algo": "scrypt",
        "salt": "00112233445566778899aabbccddeeff",
        "n": 16384,
        "r": 8,
        "p": 1,
        "hash": "862121e78531876c...0ab9cb",
    },
    "kasun": {
        "algo": "sha256",
        "hash": "82d5fb592cbf8dd48bfbbb9fa2c4075396353ef09f6ea53fae8bfefbba87a465",
    },
}
```

  The scrypt hash is shortened here. The test plan gives it in full.
- Every call to `hash_password` makes a new salt.
- A failed login leaves the users unchanged. A successful login on a scrypt record leaves it unchanged.
- The functions neither print nor read input.

- Importing the file reads no input and prints nothing.

Design

The following rules say how to write the code. Follow them.

- Compare two hashes with `hmac.compare_digest`.
- Give every variable and function a type hint.

Input validation

Not required. Assume every record is a scrypt record or a legacy record in the shapes above.

Test plan

`fixed_record` is the following record of `"lotus-2026"`. Its 128-character hash is split over two string lines, which Python joins into one string.

```
{
    "algo": "scrypt",
    "salt": "00112233445566778899aabbccddeeff",
    "n": 16384,
    "r": 8,
    "p": 1,
    "hash": (
        "862121e78531876c6ce4cb1a10fb4ff7f6c0f8c2925dac70ac036a17a8234401"
        "2d992e2b4af0bdad748cac41e5f6caaa473640d1e08f12ca38463e327a0ab9cb"
    ),
}
```

`legacy_record` is `{"algo": "sha256", "hash": nimali_hash}`, also of `"lotus-2026"`, with `nimali_hash` as in Core part A. Each `users` starts fresh in each test.

Test	Call	Expected output	Actual output	Pass
1	`hash_password("lotus-2026")`	A dictionary with exactly the keys `algo`, `salt`, `n`, `r`, `p` and `hash`. `algo` is `"scrypt"`, `n` is `16384`, `r` is `8` and `p` is `1`		
2	`hash_password("lotus-2026")`	`salt` is 32 hex characters and `hash` is 128 hex characters		
3	`hash_password("same")` twice	The two salts differ, and the two hashes differ		
4	`verify_password("lotus-2026", fixed_record)`	`True`		
5	`verify_password("Lotus-2026", fixed_record)`	`False`		
6	`verify_password("lotus-2026", hash_password("lotus-2026"))`	`True`		
7	`login("nimali", "lotus-2026", users)`, with `users` as `{"nimali": legacy_record}`	`True`. `users["nimali"]` is now a scrypt record, and `verify_password("lotus-2026", users["nimali"])` is `True`		
8	`login("nimali", "Lotus-2026", users)`, with `users` as `{"nimali": legacy_record}`	`False`. `users` is unchanged		
9	`login("nimali", "lotus-2026", users)`, with `users` as `{"nimali": fixed_record}`	`True`. `users` is unchanged		
10	`login("nimali", "Lotus-2026", users)`, with `users` as `{"nimali": fixed_record}`	`False`. `users` is unchanged		
11	`login("kasun", "lotus-2026", users)`, with `users` as `{"nimali": fixed_record}`	`False`. `users` is unchanged		
12	`import passwords`	Reads no input and prints nothing		

Done criteria

☐ Every test plan row gives exactly the expected result.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM on the day before the next session.
- The submission is one GitHub link to the fork, posted on the portal.